

Name: **Deema Mohammed AL-Maqadma** ID: **2320230766** Eng: **Mohammed Altahrawi**

documents في هذه الخلية نقوم بتحميل جميع الملفات النصية الموجودة داخل مجلد
TF-IDF لاستخدامه لاحقاً في المعالجة وبناء نموذج documents وإضافة محتواه إلى قائمة txt. الهدف: قراءة كل ملف

```
import os
```

```
path = "/content/documents" # مسار المجلد الذي يحتوي على ملفات النصوص
documents = [] # قائمة لتخزين محتوى كل مستند
filenames = []
```

```
# قراءة الملفات النصية داخل المجلد بالترتيب
for filename in sorted(os.listdir(path)):
    with open(os.path.join(path, filename), "r", encoding="utf-8") as f:
        documents.append(f.read()) # إضافة محتوى الملف إلى القائمة
        filenames.append(filename)
```

هذه الخلية للتأكد من أن المجلد تم رفعه بشكل صحيح
documents وتعرض أسماء الملفات الموجودة داخل مجلد

```
os.listdir("/content/documents")
```

```
['doc14.txt.txt',
 'doc4.txt.txt',
 'doc18.txt.txt',
 'doc3.txt.txt',
 'doc2.txt.txt',
 'doc7.txt.txt',
 'doc8.txt.txt',
 'doc17.txt.txt',
 'doc1.txt.txt',
 'doc13.txt.txt',
 'doc11.txt.txt',
 'doc16.txt.txt',
 'doc9.txt.txt',
 'doc10.txt.txt',
 'doc6.txt.txt',
 'doc5.txt.txt',
 'doc19.txt.txt',
 'doc15.txt.txt',
 'doc12.txt.txt',
 'doc20.txt.txt']
```

Stopwords في هذه الخلية نقوم بعرض قائمة كلمات الـ
الهدف: معرفة عدد الكلمات التي سيتم حذفها أثناء عملية تنظيف النصوص
هذه الكلمات تعتبر شائعة جداً ولا تضيف معنى مهم لعملية البحث

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
```

```
# stopwords طباعة عدد كلمات الـ
print("Stopwords: عدد كلمات الـ", len(list(ENGLISH_STOP_WORDS)))
```

```
# stopwords طباعة قائمة كاملة بالـ
print("Stopwords: قائمة الـ")
print(list(ENGLISH_STOP_WORDS))
```

```
Stopwords: 318 عدد كلمات الـ
```

```
Stopwords: قائمة الـ
```

```
['less', 'mostly', 'they', 'onto', 'can', 'de', 'in', 'with', 'too', 'done', 'yet', 'perhaps', 'anywhere', 'next', 'either',
```

للنصوص preprocessing في هذه الخلية نقوم بعمل الـ
الخطوات:

```
# 1) lowercase تحويل النص إلى
# 2) إزالة علامات الترقيم
# 3) stopwords إزالة كلمات الـ
# 4) إعادة تجميع الكلمات في نص نظيف
```

```
import string
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
```

```
cleaned_docs = [] # قائمة لحفظ النصوص بعد التنظيف
```

```
# جدول لإزالة علامات الترقيم
punct_table = str.maketrans("", "", string.punctuation)
```

```
for doc in documents:
    text = doc.lower() # lowercase تحويل إلى
    text = text.translate(punct_table) # إزالة علامات الترقيم
    words = text.split() # تقسيم النص إلى كلمات
    words = [w for w in words if w not in ENGLISH_STOP_WORDS] # إزالة stopwords
```

```
wurds = [w for w in words if w not in ENGLISH_STOP_WORDS] # إزالة stopwords
cleaned_docs.append(" ".join(words)) # إعادة تجميع النص
```

```
cleaned_docs[:3] # عرض أول 3 مستندات بعد التنظيف
```

```
['artificial intelligence transforms modern industries machine learning models support medical diagnosis ai systems analyze
patterns inside large datasets smart algorithms improve decision processes automation increases efficiency sectors',
'mobile applications improve daily tasks user interfaces require intuitive design notifications users updated cloud
storage syncs personal files app performance affects user experience',
'digital communication connects people worldwide messaging apps support instant interaction video calls enable remote
collaboration social networks share information quickly online communities grow rapidly']
```

```
# TF-IDF في هذه الخلية نقوم ببناء نموذج
```

```
# هذا النموذج يحول النصوص إلى تمثيل عددي يعتمد على أهمية الكلمات
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
vectorizer = TfidfVectorizer() # إنشاء النموذج
```

```
tfidf_matrix = vectorizer.fit_transform(cleaned_docs) # تدريب النموذج على المستندات
```

```
tfidf_matrix # عرض المصفوفة الناتجة
```

```
<Compressed Sparse Row sparse matrix of dtype 'float64'
with 463 stored elements and shape (20, 365)>
```

```
# (vocabulary) في هذه الخلية نعرض قائمة الكلمات
```

```
# TF-IDF وهي الكلمات التي اعتمد عليها نموذج
```

```
vocabulary = vectorizer.get_feature_names_out()
```

```
vocabulary
```

```
'collaboration', 'commerce', 'communication', 'communities',
'competitive', 'complex', 'components', 'computer', 'conditions',
'connects', 'consumption', 'control', 'cooperation', 'coordinate',
'cosmic', 'crop', 'customer', 'customers', 'cybersecurity',
'daily', 'damage', 'data', 'datasets', 'decision', 'decisions',
'deep', 'delivery', 'design', 'developers', 'development',
'devices', 'diagnosis', 'digital', 'diseases', 'distant',
'doctors', 'documentation', 'drivers', 'drives', 'drones',
'easily', 'eco', 'ecosystems', 'education', 'effectively',
'efficiency', 'efficiently', 'electric', 'electricity',
'embeddings', 'emissions', 'emotions', 'enable', 'encryption',
'energy', 'engineering', 'enhances', 'ensures', 'environmental',
'equipment', 'error', 'events', 'evolve', 'expands', 'experience',
'explores', 'extracts', 'extreme', 'farmers', 'farming', 'fast',
'field', 'files', 'financial', 'fintech', 'firewalls', 'flow',
'follow', 'frequent', 'friendly', 'fuel', 'gain', 'galaxies',
'gateways', 'generate', 'global', 'goals', 'good', 'green', 'grow',
'growth', 'guide', 'habitats', 'handles', 'healthcare', 'healthy',
'hidden', 'hospitals', 'human', 'image', 'imaging', 'impact',
'improve', 'improves', 'increases', 'industrial', 'industries',
'influence', 'influences', 'information', 'innovate', 'innovation',
'inside', 'insights', 'instant', 'intelligence', 'interaction',
'interfaces', 'international', 'intuitive', 'invest', 'irrigation',
'knowledge', 'language', 'large', 'leaders', 'learning',
'lifelong', 'lifestyles', 'lights', 'like', 'lowers', 'machine',
'making', 'manage', 'management', 'manufacturing', 'marketing',
'meaning', 'medical', 'mental', 'messaging', 'methods', 'metrics',
'mind', 'missions', 'mobile', 'mobility', 'model', 'modeling',
'models', 'modern', 'modernizes', 'modular', 'monitor', 'movement',
'natural', 'navigation', 'networks', 'neural', 'notifications',
'objects', 'observe', 'online', 'organizational', 'organizations',
'organizes', 'panels', 'patient', 'patterns', 'payment',
'payments', 'people', 'performance', 'personal', 'planetary',
'planning', 'platforms', 'pollution', 'power', 'powers',
'practices', 'precision', 'predictive', 'prepares', 'present',
'prevent', 'processes', 'processing', 'production', 'products',
'progress', 'project', 'protects', 'psychology', 'public',
'quality', 'quickly', 'radiation', 'rapidly', 'raw', 'reactions',
'recognition', 'reduce', 'reduces', 'reliability', 'rely',
'remote', 'renewable', 'repetitive', 'require', 'requires',
'research', 'resources', 'responses', 'results', 'reveal',
'rising', 'road', 'robotic', 'robotics', 'robots', 'safe', 'sales',
'satisfaction', 'saves', 'science', 'scientists', 'sectors',
'secure', 'security', 'semantic', 'sensitive', 'sensors',
'sentences', 'service', 'services', 'set', 'shape', 'share',
'shopping', 'simplify', 'smart', 'social', 'software', 'soil',
```

```
visualization', wallets, water, weather, well, wildlife,
'wind', 'wood', 'workflow', 'worldwide', 'wild', dtype=object)
```

```
# في هذه الخلية نحول مصفوفة (DataFrame) إلى جدول TF-IDF
# لعرض القيم بشكل أوضح

import pandas as pd

tfidf_dense = tfidf_matrix.toarray() # تحويل إلى مصفوفة كثيفة
tfidf_df = pd.DataFrame(tfidf_dense, columns=vocabulary)

tfidf_df.head() # عرض أول 5 صفوف
```

	academic	access	accuracy	accurate	accurately	action	adopt	advanced	advantage	affects	...	wallets	water	wea
0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.000000	...	0.0	0.000000	
1	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.188707	...	0.0	0.000000	
2	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.000000	...	0.0	0.000000	
3	0.0	0.0	0.0	0.0	0.210857	0.0	0.0	0.0	0.0	0.000000	...	0.0	0.000000	
4	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.000000	...	0.0	0.218072	

5 rows × 365 columns

```
# في هذه الخلية ندخل استعمال المستخدم
# preprocessing ونطبق عليه نفس خطوات الـ

query = input("Enter your search query: ")

# تنظيف الاستعلام
q = query.lower()
q = q.translate(punct_table)
q_words = q.split()
q_words = [w for w in q_words if w not in ENGLISH_STOP_WORDS]
cleaned_query = " ".join(q_words)

# تحويل الاستعلام إلى متجه TF-IDF
query_vec = vectorizer.transform([cleaned_query])

query_vec
```

```
Enter your search query: computer science in the information technology
<Compressed Sparse Row sparse matrix of dtype 'float64'
  with 4 stored elements and shape (1, 365)>
```

```
# في هذه الخلية نحسب درجة التشابه بين الاستعلام وكل المستندات
# باستخدام Cosine Similarity

from sklearn.metrics.pairwise import cosine_similarity

similarities = cosine_similarity(query_vec, tfidf_matrix)
similarities

array([[0.          , 0.          , 0.09240568, 0.11432322, 0.07438482,
        0.          , 0.          , 0.          , 0.07962819, 0.          ,
        0.          , 0.09123524, 0.          , 0.07012972, 0.          ,
        0.          , 0.          , 0.          , 0.          , 0.10460009]])
```

```
# في هذه الخلية نرتب المستندات حسب أعلى درجة تشابه
# ثم نعرض النتائج من الأعلى إلى الأقل

import numpy as np

ranking = np.argsort(similarities[0])[::-1] # ترتيب تنازلي

for idx in ranking:
    print(f"Score: {similarities[0][idx]:.4f} | Document: {filenames[idx]}")
```

```
Score: 0.1143 | Document: doc12.txt.txt
Score: 0.1046 | Document: doc9.txt.txt
Score: 0.0924 | Document: doc11.txt.txt
Score: 0.0912 | Document: doc2.txt.txt
Score: 0.0796 | Document: doc17.txt.txt
Score: 0.0744 | Document: doc13.txt.txt
Score: 0.0701 | Document: doc3.txt.txt
Score: 0.0000 | Document: doc8.txt.txt
Score: 0.0000 | Document: doc6.txt.txt
Score: 0.0000 | Document: doc7.txt.txt
Score: 0.0000 | Document: doc20.txt.txt
Score: 0.0000 | Document: doc5.txt.txt
Score: 0.0000 | Document: doc18.txt.txt
```

Score: 0.0000	Document: doc19.txt.txt
Score: 0.0000	Document: doc4.txt.txt
Score: 0.0000	Document: doc16.txt.txt
Score: 0.0000	Document: doc14.txt.txt
Score: 0.0000	Document: doc15.txt.txt
Score: 0.0000	Document: doc10.txt.txt
Score: 0.0000	Document: doc1.txt.txt

"وأخر دعواهم أن الحمد لله رب العالمين"